

# Computer Science: The Part They Expect You to Just Know

*Python, Data Structures, and Algorithms for When the Interview Is Tomorrow*

Alan Li

New York University

# Contents

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| <b>I</b>  | <b>Programming Language</b>                        | <b>1</b>  |
| <b>1</b>  | <b>Python</b>                                      | <b>2</b>  |
| 1.1       | Data Types & Representation . . . . .              | 2         |
| 1.2       | Operators & Expressions . . . . .                  | 3         |
| 1.3       | Control Flow . . . . .                             | 4         |
| 1.4       | Functions . . . . .                                | 5         |
| 1.5       | Strings . . . . .                                  | 6         |
| 1.6       | Lists & Tuples . . . . .                           | 7         |
| 1.7       | Dictionaries & Sets . . . . .                      | 8         |
| 1.8       | File I/O & Exceptions . . . . .                    | 10        |
| 1.9       | Built-in Functions & Modules . . . . .             | 11        |
| <b>II</b> | <b>Data Structures &amp; Algorithms</b>            | <b>13</b> |
| <b>2</b>  | <b>Foundations</b>                                 | <b>14</b> |
| 2.1       | Algorithm Analysis & Asymptotic Notation . . . . . | 14        |
| 2.2       | Recursion . . . . .                                | 15        |
| <b>3</b>  | <b>Core Data Structures</b>                        | <b>16</b> |
| 3.1       | Arrays . . . . .                                   | 16        |
| 3.2       | Stacks . . . . .                                   | 16        |
| 3.3       | Queues . . . . .                                   | 17        |
| 3.4       | Linked Lists . . . . .                             | 17        |
| 3.5       | Trees . . . . .                                    | 18        |
| <b>4</b>  | <b>Sorting</b>                                     | <b>19</b> |
| 4.1       | Comparison-Based Sorting Algorithms . . . . .      | 19        |
| 4.2       | Lower Bound for Comparison Sorting . . . . .       | 21        |
| 4.3       | Sorting Algorithms Summary . . . . .               | 21        |
| <b>5</b>  | <b>Graph Algorithms</b>                            | <b>22</b> |
| 5.1       | Graph Basics . . . . .                             | 22        |
| 5.2       | Breadth-First Search (BFS) . . . . .               | 22        |
| 5.3       | Depth-First Search (DFS) & Applications . . . . .  | 23        |
| 5.4       | Topological Sort . . . . .                         | 24        |
| 5.5       | Strongly Connected Components (SCC) . . . . .      | 24        |
| 5.6       | Minimum Spanning Trees (MST) . . . . .             | 24        |
| 5.7       | Shortest Paths . . . . .                           | 25        |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>6</b> | <b>Algorithm Design Paradigms</b>     | <b>27</b> |
| 6.1      | Divide & Conquer . . . . .            | 27        |
| 6.2      | Greedy Algorithms . . . . .           | 27        |
| 6.3      | Dynamic Programming . . . . .         | 28        |
| 6.4      | Matching . . . . .                    | 29        |
| <b>7</b> | <b>Computability &amp; Complexity</b> | <b>30</b> |
| 7.1      | Computability . . . . .               | 30        |
| 7.2      | NP-Completeness . . . . .             | 31        |

Part I

Programming Language

# Chapter 1

# Python

Python is a high-level, interpreted, dynamically-typed language. This chapter is a structured reference for core language features, organized for quick lookup. Code examples use Python 3.

## 1.1 Data Types & Representation

### Primitive Types

| Type     | Keyword              | Description                                                                | Example                                |
|----------|----------------------|----------------------------------------------------------------------------|----------------------------------------|
| Integer  | <code>int</code>     | Arbitrary precision whole number                                           | 42, -7, 0b1010, 0xFF                   |
| Float    | <code>float</code>   | 64-bit IEEE 754 double                                                     | 3.14, 2.5e-3                           |
| Complex  | <code>complex</code> | Real + imaginary part                                                      | 3+4j, <code>complex(1,2)</code>        |
| Boolean  | <code>bool</code>    | Subclass of <code>int</code> ; <code>True</code> =1, <code>False</code> =0 | <code>True</code> , <code>False</code> |
| String   | <code>str</code>     | Immutable Unicode sequence                                                 | "hello", 'world'                       |
| NoneType | <code>None</code>    | Null/absence of value                                                      | <code>None</code>                      |

#### Remark.

Integers in Python have arbitrary precision — no integer overflow. Booleans are a subclass of `int`: `True + True == 2`.

### Type Conversion Functions

| Function                  | Converts to   | Notes                                                              |
|---------------------------|---------------|--------------------------------------------------------------------|
| <code>int(x)</code>       | Integer       | <code>int(3.9) → 3</code> (truncates); <code>int("42") → 42</code> |
| <code>float(x)</code>     | Float         | <code>float("3.14") → 3.14</code>                                  |
| <code>str(x)</code>       | String        | <code>str(42) → "42"</code>                                        |
| <code>bool(x)</code>      | Boolean       | <code>False</code> for: 0, "", [], {}, <code>None</code>           |
| <code>complex(r,i)</code> | Complex       | <code>complex(3,4) → (3+4j)</code>                                 |
| <code>bin(n)</code>       | Binary string | <code>bin(10) → "0b1010"</code>                                    |
| <code>hex(n)</code>       | Hex string    | <code>hex(255) → "0xff"</code>                                     |
| <code>ord(c)</code>       | Unicode int   | <code>ord("A") → 65</code>                                         |
| <code>chr(n)</code>       | Character     | <code>chr(65) → "A"</code>                                         |

# Data Representation

## Definition 1.1.1: Bit Width

An  $n$ -bit value can represent  $2^n$  distinct states.

| Bits | Range (unsigned) | Common use           |
|------|------------------|----------------------|
| 1    | 0–1              | boolean flag         |
| 8    | 0–255            | byte, ASCII char     |
| 16   | 0–65 535         | short integer        |
| 32   | 0–4 294 967 295  | int (C)              |
| 64   | 0– $2^{64} - 1$  | long, float (Python) |

# 1.2 Operators & Expressions

## Arithmetic Operators

| Op | Name           | Example                                           |
|----|----------------|---------------------------------------------------|
| +  | Addition       | $3 + 4 \rightarrow 7$                             |
| -  | Subtraction    | $7 - 3 \rightarrow 4$                             |
| *  | Multiplication | $3 * 4 \rightarrow 12$                            |
| /  | True division  | $7 / 2 \rightarrow 3.5$                           |
| // | Floor division | $7 // 2 \rightarrow 3$ ; $-7 // 2 \rightarrow -4$ |
| %  | Modulo         | $7 \% 3 \rightarrow 1$                            |
| ** | Exponentiation | $2 ** 3 \rightarrow 8$                            |

## Comparison & Logical Operators

| Op     | Name                  | Notes                                                                    |
|--------|-----------------------|--------------------------------------------------------------------------|
| ==     | Equal                 | Value equality                                                           |
| !=     | Not equal             |                                                                          |
| <      | Less than             |                                                                          |
| >      | Greater than          |                                                                          |
| <=     | Less than or equal    |                                                                          |
| >=     | Greater than or equal |                                                                          |
| and    | Logical AND           | Short-circuit: returns first falsy value or last value                   |
| or     | Logical OR            | Short-circuit: returns first truthy value or last value                  |
| not    | Logical NOT           | Unary; <code>not True</code> $\rightarrow$ <code>False</code>            |
| in     | Membership            | <code>"a" in "abc"</code> $\rightarrow$ <code>True</code>                |
| not in | Non-membership        |                                                                          |
| is     | Identity              | Same object in memory (not same value)                                   |
| is not | Non-identity          | Use <code>is</code> for <code>None</code> checks: <code>x is None</code> |

## Compound Assignment Operators

| Op                  | Equivalent             | Op                                                    |
|---------------------|------------------------|-------------------------------------------------------|
| <code>x += n</code> | <code>x = x + n</code> | <code>x /= n</code> $\equiv$ <code>x = x / n</code>   |
| <code>x -= n</code> | <code>x = x - n</code> | <code>x %= n</code> $\equiv$ <code>x = x % n</code>   |
| <code>x *= n</code> | <code>x = x * n</code> | <code>x **= n</code> $\equiv$ <code>x = x ** n</code> |
| <code>x /= n</code> | <code>x = x / n</code> |                                                       |

**Remark.**

**Operator precedence** (high to low): `**` > unary `-/+` > `*` / `//` / `%` > `+` / `-` > `<` / `>` / `<=` / `>=` > `==` / `!=` > `not` > `and` > `or`. Use parentheses when in doubt.

## 1.3 Control Flow

### Conditionals

```
if condition:
    ...
elif another_condition:
    ...
else:
    ...

# Ternary expression
value = x if condition else y
```

### While Loop

```
while condition:
    ...
    if some_case:
        break          # exit loop immediately
    if other_case:
        continue       # skip to next iteration
else:
    ...                 # runs if loop completed without break
```

### For Loop

```
for item in iterable:
    ...

for i in range(start, stop, step):    # range(5) = 0,1,2,3,4
    ...
```

```

for i, val in enumerate(seq, start=0):
    ...

for a, b in zip(seq1, seq2):           # parallel iteration
    ...

```

## Comprehensions

| Type      | Syntax                                    | Returns          |
|-----------|-------------------------------------------|------------------|
| List      | <code>[expr for x in iter if cond]</code> | list             |
| Dict      | <code>{k: v for x in iter if cond}</code> | dict             |
| Set       | <code>{expr for x in iter if cond}</code> | set              |
| Generator | <code>(expr for x in iter if cond)</code> | generator (lazy) |

### Example 1.3.1

```

squares = [x**2 for x in range(10)] → [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
even_sq = {x**2 for x in range(10) if x % 2 == 0} → {0, 4, 16, 36, 64}

```

## 1.4 Functions

### Definition 1.4.1: Function

A reusable, named block of code defined with **def**. Returns **None** implicitly if no **return** statement is reached. Multiple values can be returned as a tuple.

### Parameter Types

| Type                | Syntax                       | Example call                                   |
|---------------------|------------------------------|------------------------------------------------|
| Positional          | <code>def f(a, b)</code>     | <code>f(1, 2)</code>                           |
| Default value       | <code>def f(a, b=0)</code>   | <code>f(1)</code>                              |
| Variable positional | <code>def f(*args)</code>    | <code>f(1,2,3) ⇒ args=(1,2,3)</code>           |
| Variable keyword    | <code>def f(**kwargs)</code> | <code>f(x=1,y=2) ⇒ kwargs={'x':1,'y':2}</code> |
| Keyword-only        | <code>def f(*, a)</code>     | <code>f(a=1)</code> (must be named)            |
| Positional-only     | <code>def f(a, b, /)</code>  | <code>f(1, 2)</code> (cannot be named)         |

#### Remark.

Order of parameters: positional → default → **\*args** → keyword-only → **\*\*kwargs**.



## Scope: LEGB Rule

| Level     | Description                        | Example                                                    |
|-----------|------------------------------------|------------------------------------------------------------|
| Local     | Inside the current function        | variable defined in <code>def</code>                       |
| Enclosing | Outer function in nested functions | closure variable                                           |
| Global    | Module-level (top of file)         | <code>global x</code> to write                             |
| Built-in  | Python's pre-defined names         | <code>len</code> , <code>print</code> , <code>range</code> |

## Lambda Functions

```
f = lambda x, y: x + y          # anonymous function, single expression
sorted(items, key=lambda x: x[1])
```

## 1.5 Strings

Strings are *immutable* sequences of Unicode characters. Indexing uses `s[i]` (0-based); slicing uses `s[start:stop:step]`.

### String Methods — Case

| Method                     | Description           | Example                                            |
|----------------------------|-----------------------|----------------------------------------------------|
| <code>.lower()</code>      | All lowercase         | <code>"Hello".lower() → "hello"</code>             |
| <code>.upper()</code>      | All uppercase         | <code>"hello".upper() → "HELLO"</code>             |
| <code>.title()</code>      | Title Case            | <code>"hello world".title() → "Hello World"</code> |
| <code>.capitalize()</code> | Capitalize first char | <code>"hello".capitalize() → "Hello"</code>        |
| <code>.swapcase()</code>   | Swap all cases        | <code>"Hello".swapcase() → "hELLO"</code>          |

### String Methods — Search & Check

| Method                      | Description                                                  | Returns           |
|-----------------------------|--------------------------------------------------------------|-------------------|
| <code>.find(s)</code>       | Index of first <code>s</code> ; <code>-1</code> if not found | <code>int</code>  |
| <code>.rfind(s)</code>      | Index of last <code>s</code>                                 | <code>int</code>  |
| <code>.index(s)</code>      | Like <code>find</code> but raises <code>ValueError</code>    | <code>int</code>  |
| <code>.count(s)</code>      | Count non-overlapping occurrences of <code>s</code>          | <code>int</code>  |
| <code>.startswith(s)</code> | True if string starts with <code>s</code>                    | <code>bool</code> |
| <code>.endswith(s)</code>   | True if string ends with <code>s</code>                      | <code>bool</code> |
| <code>.isdigit()</code>     | All chars are digits                                         | <code>bool</code> |
| <code>.isalpha()</code>     | All chars are letters                                        | <code>bool</code> |
| <code>.isalnum()</code>     | All chars are letters or digits                              | <code>bool</code> |
| <code>.isspace()</code>     | All chars are whitespace                                     | <code>bool</code> |
| <code>.isupper()</code>     | All cased chars are uppercase                                | <code>bool</code> |
| <code>.islower()</code>     | All cased chars are lowercase                                | <code>bool</code> |

## String Methods — Strip & Replace

| Method                             | Description                                                       |
|------------------------------------|-------------------------------------------------------------------|
| <code>.strip()</code>              | Remove leading and trailing whitespace                            |
| <code>.lstrip()</code>             | Remove leading whitespace                                         |
| <code>.rstrip()</code>             | Remove trailing whitespace                                        |
| <code>.strip(chars)</code>         | Remove any chars in <code>chars</code> from both ends             |
| <code>.replace(old, new)</code>    | Replace all occurrences of <code>old</code> with <code>new</code> |
| <code>.replace(old, new, n)</code> | Replace at most <code>n</code> occurrences                        |

## String Methods — Split & Join

| Method                      | Description                         | Example                                        |
|-----------------------------|-------------------------------------|------------------------------------------------|
| <code>.split()</code>       | Split on whitespace                 | <code>"a b c".split() → ['a', 'b', 'c']</code> |
| <code>.split(sep)</code>    | Split on <code>sep</code>           | <code>"a,b".split(",") → ['a', 'b']</code>     |
| <code>.split(sep, n)</code> | Split at most <code>n</code> times  |                                                |
| <code>.rsplit(sep)</code>   | Split from right                    |                                                |
| <code>.splitlines()</code>  | Split on line boundaries            |                                                |
| <code>sep.join(iter)</code> | Join iterable with <code>sep</code> | <code>", ".join(['a', 'b']) → "a, b"</code>    |

## String Formatting

| Style                  | Syntax                                    | Example                    |
|------------------------|-------------------------------------------|----------------------------|
| % formatting           | <code>"%s is %d" % (name, age)</code>     | <code>"Alice is 25"</code> |
| <code>.format()</code> | <code>"{} is {}".format(name, age)</code> | <code>"Alice is 25"</code> |
| f-string (3.6+)        | <code>f"{name} is {age}"</code>           | <code>"Alice is 25"</code> |

| Format spec          | Meaning             | Example                                    |
|----------------------|---------------------|--------------------------------------------|
| <code>:.2f</code>    | 2 decimal places    | <code>f"{3.14159:.2f}" → "3.14"</code>     |
| <code>:10</code>     | Width 10            | <code>f"{42:10}" → " 42"</code>            |
| <code>:&lt;10</code> | Left-align width 10 | <code>f"{42:&lt;10}" → "42 "</code>        |
| <code>:^10</code>    | Center width 10     |                                            |
| <code>:,</code>      | Thousands separator | <code>f"{1000000:,}" → "1,000,000"</code>  |
| <code>:e</code>      | Scientific notation | <code>f"{0.001:e}" → "1.000000e-03"</code> |

## 1.6 Lists & Tuples

### Definition 1.6.1: List

An ordered, *mutable* sequence. Elements can be of mixed types. Backed by a dynamic array; random access in  $O(1)$ , append in amortized  $O(1)$ .

## List Methods

| Method                           | Description                                       | Time          |
|----------------------------------|---------------------------------------------------|---------------|
| <code>.append(x)</code>          | Add <code>x</code> to end                         | $O(1)^*$      |
| <code>.extend(iter)</code>       | Add all items from iterable                       | $O(k)$        |
| <code>.insert(i, x)</code>       | Insert <code>x</code> at index <code>i</code>     | $O(n)$        |
| <code>.remove(x)</code>          | Remove first occurrence of <code>x</code>         | $O(n)$        |
| <code>.pop()</code>              | Remove and return last element                    | $O(1)^*$      |
| <code>.pop(i)</code>             | Remove and return element at index <code>i</code> | $O(n)$        |
| <code>.index(x)</code>           | Index of first occurrence of <code>x</code>       | $O(n)$        |
| <code>.count(x)</code>           | Count occurrences of <code>x</code>               | $O(n)$        |
| <code>.sort(key, reverse)</code> | Sort in place (Timsort)                           | $O(n \log n)$ |
| <code>.reverse()</code>          | Reverse in place                                  | $O(n)$        |
| <code>.copy()</code>             | Shallow copy                                      | $O(n)$        |
| <code>.clear()</code>            | Remove all items                                  | $O(n)$        |

### Remark.

\* Amortized. Use `sorted(lst)` to get a new sorted list without modifying the original. `lst[start:stop:step]` slices return a new list.

## List vs. Tuple

| Feature     | List                   | Tuple                                              |
|-------------|------------------------|----------------------------------------------------|
| Syntax      | <code>[1, 2, 3]</code> | <code>(1, 2, 3)</code>                             |
| Mutable     | Yes                    | No                                                 |
| Hashable    | No                     | Yes (if all elements hashable)                     |
| Performance | Slightly slower        | Slightly faster                                    |
| Typical use | Mutable collections    | Fixed records, dict keys, multiple return values   |
| Methods     | 11 methods             | Only <code>.count()</code> , <code>.index()</code> |

### Remark.

A single-element tuple requires a trailing comma: `(1,)` not `(1)`.

## 1.7 Dictionaries & Sets

### Definition 1.7.1: Dictionary

An unordered (insertion-ordered since Python 3.7) mapping of *hashable* keys to values. Implemented as a hash table; average  $O(1)$  access, insertion, and deletion.

## Dictionary Methods

| Method / Operation                      | Description                                          |
|-----------------------------------------|------------------------------------------------------|
| <code>d[key]</code>                     | Access value; raises <code>KeyError</code> if absent |
| <code>d.get(key)</code>                 | Access value; returns <code>None</code> if absent    |
| <code>d.get(key, default)</code>        | Access value with fallback                           |
| <code>d[key] = val</code>               | Set or update key                                    |
| <code>del d[key]</code>                 | Delete key                                           |
| <code>key in d</code>                   | Membership test for key                              |
| <code>d.keys()</code>                   | View of all keys                                     |
| <code>d.values()</code>                 | View of all values                                   |
| <code>d.items()</code>                  | View of all (key, value) pairs                       |
| <code>d.update(other)</code>            | Merge <code>other</code> dict into <code>d</code>    |
| <code>d.pop(key)</code>                 | Remove key and return its value                      |
| <code>d.pop(key, default)</code>        | Remove key with fallback                             |
| <code>d.popitem()</code>                | Remove and return last (key, val) pair               |
| <code>d.setdefault(key, default)</code> | Get value; set to default if key absent              |
| <code>d.clear()</code>                  | Remove all items                                     |
| <code>d.copy()</code>                   | Shallow copy                                         |
| <code>d   other</code>                  | Merge (Python 3.9+); creates new dict                |

### Definition 1.7.2: Set

An unordered collection of unique, hashable elements. Backed by a hash table; average  $O(1)$  insertion, deletion, and membership test. `set()` creates an empty set; `{}` creates an empty *dict*.

## Set Operations

| Operator     | Method                                 | Description                       |
|--------------|----------------------------------------|-----------------------------------|
| $A \mid B$   | <code>A.union(B)</code>                | All elements in $A$ or $B$        |
| $A \& B$     | <code>A.intersection(B)</code>         | Elements in both $A$ and $B$      |
| $A - B$      | <code>A.difference(B)</code>           | Elements in $A$ but not $B$       |
| $A \wedge B$ | <code>A.symmetric_difference(B)</code> | Elements in exactly one of $A, B$ |
| $A \leq B$   | <code>A.issubset(B)</code>             | $A \subseteq B$                   |
| $A \geq B$   | <code>A.issuperset(B)</code>           | $A \supseteq B$                   |
| $A < B$      | —                                      | $A \subsetneq B$ (proper subset)  |

## Set Methods

| Method                     | Description                                                    |
|----------------------------|----------------------------------------------------------------|
| <code>.add(x)</code>       | Add <code>x</code> ; no-op if already present                  |
| <code>.remove(x)</code>    | Remove <code>x</code> ; raises <code>KeyError</code> if absent |
| <code>.discard(x)</code>   | Remove <code>x</code> ; no-op if absent                        |
| <code>.pop()</code>        | Remove and return an arbitrary element                         |
| <code>.update(iter)</code> | Add all elements from iterable                                 |
| <code>.clear()</code>      | Remove all elements                                            |
| <code>.copy()</code>       | Shallow copy                                                   |

## 1.8 File I/O & Exceptions

### File Modes

| Mode | Name       | Behaviour                                                       |
|------|------------|-----------------------------------------------------------------|
| 'r'  | Read       | Default. Fails if file doesn't exist.                           |
| 'w'  | Write      | Creates file or truncates existing.                             |
| 'a'  | Append     | Creates file or appends to existing.                            |
| 'x'  | Create     | Creates file; raises <code>FileExistsError</code> if it exists. |
| 'b'  | Binary     | Modifier; e.g. 'rb', 'wb'.                                      |
| '+'  | Read+Write | Modifier; e.g. 'r+', 'w+'.                                      |

### File Methods

| Method                          | Description                                                   |
|---------------------------------|---------------------------------------------------------------|
| <code>.read()</code>            | Read entire file as one string                                |
| <code>.read(n)</code>           | Read at most <code>n</code> characters                        |
| <code>.readline()</code>        | Read one line (including <code>\n</code> )                    |
| <code>.readlines()</code>       | Read all lines into a list                                    |
| <code>.write(s)</code>          | Write string <code>s</code> ; returns number of chars written |
| <code>.writelines(lines)</code> | Write list of strings (no newlines added)                     |
| <code>.close()</code>           | Close the file (always call or use <code>with</code> )        |
| <code>.seek(pos)</code>         | Move file pointer to byte position                            |
| <code>.tell()</code>            | Return current file pointer position                          |

#### Remark.

Use the `with` statement — it guarantees the file is closed even if an exception occurs: `with open("file.txt") as f: ...`

### Exception Handling

```

try:
    risky_operation()
except ValueError as e:           # catch specific exception
    handle(e)
except (TypeError, KeyError):    # catch multiple types
    ...
except Exception as e:          # catch all (avoid)
    ...
else:
    ...                          # runs only if NO exception was raised
finally:
    ...                          # always runs (cleanup)

```

## Common Built-in Exceptions

| Exception         | Raised when                                                |
|-------------------|------------------------------------------------------------|
| ValueError        | Correct type but invalid value ( <code>int("abc")</code> ) |
| TypeError         | Wrong type ( <code>"1" + 1</code> )                        |
| IndexError        | Sequence index out of range                                |
| KeyError          | Dict key not found                                         |
| AttributeError    | Object has no such attribute                               |
| NameError         | Variable or name not defined                               |
| ZeroDivisionError | Division or modulo by zero                                 |
| FileNotFoundError | File or directory not found                                |
| ImportError       | Module cannot be imported                                  |
| StopIteration     | Iterator is exhausted                                      |
| RecursionError    | Maximum recursion depth exceeded                           |

## 1.9 Built-in Functions & Modules

### Frequently Used Built-ins

| Function                            | Description                                           | Example                                     |
|-------------------------------------|-------------------------------------------------------|---------------------------------------------|
| <code>len(x)</code>                 | Length of sequence/collection                         | <code>len([1,2,3]) → 3</code>               |
| <code>range(stop)</code>            | Range <code>[0, stop)</code>                          | <code>range(5)</code>                       |
| <code>range(start,stop,step)</code> | Range with step                                       | <code>range(0,10,2)</code>                  |
| <code>type(x)</code>                | Type of <code>x</code>                                | <code>type(42) → &lt;class 'int'&gt;</code> |
| <code>isinstance(x, T)</code>       | Check if <code>x</code> is instance of <code>T</code> | <code>isinstance(42, int)</code>            |
| <code>abs(x)</code>                 | Absolute value                                        | <code>abs(-5) → 5</code>                    |
| <code>round(x, n)</code>            | Round to <code>n</code> decimal places                | <code>round(3.14159, 2) → 3.14</code>       |
| <code>min(*args)</code>             | Minimum value                                         | <code>min(3,1,2) → 1</code>                 |
| <code>max(*args)</code>             | Maximum value                                         | <code>max(3,1,2) → 3</code>                 |
| <code>sum(iter)</code>              | Sum of iterable                                       | <code>sum([1,2,3]) → 6</code>               |
| <code>sorted(iter)</code>           | New sorted list                                       | <code>sorted([3,1,2]) → [1,2,3]</code>      |
| <code>reversed(iter)</code>         | Reversed iterator                                     |                                             |
| <code>enumerate(iter)</code>        | ( <code>index, value</code> ) pairs                   |                                             |
| <code>zip(*iters)</code>            | Parallel iteration tuples                             |                                             |
| <code>map(f, iter)</code>           | Apply <code>f</code> to each element                  |                                             |
| <code>filter(f, iter)</code>        | Elements where <code>f</code> is truthy               |                                             |
| <code>any(iter)</code>              | True if any element is truthy                         |                                             |
| <code>all(iter)</code>              | True if all elements are truthy                       |                                             |
| <code>id(x)</code>                  | Memory address of object                              |                                             |
| <code>hash(x)</code>                | Hash value (for hashable objects)                     |                                             |

**math Module**

| Name                             | Description                   | Value / Example                      |
|----------------------------------|-------------------------------|--------------------------------------|
| <code>math.pi</code>             | $\pi$                         | $\approx 3.14159$                    |
| <code>math.e</code>              | $e$                           | $\approx 2.71828$                    |
| <code>math.inf</code>            | Positive infinity             |                                      |
| <code>math.sqrt(x)</code>        | $\sqrt{x}$                    | <code>math.sqrt(9) → 3.0</code>      |
| <code>math.floor(x)</code>       | $\lfloor x \rfloor$           | <code>math.floor(3.9) → 3</code>     |
| <code>math.ceil(x)</code>        | $\lceil x \rceil$             | <code>math.ceil(3.1) → 4</code>      |
| <code>math.log(x)</code>         | $\ln x$                       | natural log                          |
| <code>math.log(x, b)</code>      | $\log_b x$                    | <code>math.log(8, 2) → 3.0</code>    |
| <code>math.log2(x)</code>        | $\log_2 x$                    |                                      |
| <code>math.log10(x)</code>       | $\log_{10} x$                 |                                      |
| <code>math.exp(x)</code>         | $e^x$                         |                                      |
| <code>math.pow(x, y)</code>      | $x^y$ (returns float)         |                                      |
| <code>math.factorial(n)</code>   | $n!$                          | <code>math.factorial(5) → 120</code> |
| <code>math.gcd(a, b)</code>      | Greatest common divisor       | <code>math.gcd(12, 8) → 4</code>     |
| <code>math.cos/sin/tan(x)</code> | Trig functions (radians)      |                                      |
| <code>math.degrees(r)</code>     | Radians $\rightarrow$ degrees |                                      |
| <code>math.radians(d)</code>     | Degrees $\rightarrow$ radians |                                      |

**random Module**

| Function                              | Description                                    |
|---------------------------------------|------------------------------------------------|
| <code>random.random()</code>          | Float in $[0.0, 1.0)$                          |
| <code>random.uniform(a, b)</code>     | Float in $[a, b]$                              |
| <code>random.randint(a, b)</code>     | Integer in $[a, b]$ (inclusive both ends)      |
| <code>random.randrange(n)</code>      | Integer in $[0, n)$                            |
| <code>random.choice(seq)</code>       | Random element from non-empty sequence         |
| <code>random.choices(seq, k=n)</code> | $n$ random elements with replacement           |
| <code>random.sample(seq, k)</code>    | $k$ unique random elements without replacement |
| <code>random.shuffle(seq)</code>      | Shuffle sequence in place                      |
| <code>random.seed(n)</code>           | Set random seed for reproducibility            |

## Part II

# Data Structures & Algorithms



# Chapter 2

## Foundations

### 2.1 Algorithm Analysis & Asymptotic Notation

#### Definition 2.1.1: Asymptotic Upper Bound ( $O$ )

$f(n) = O(g(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 1$  such that for all  $n \geq n_0$ :

$$f(n) \leq c \cdot g(n)$$

Informally:  $f$  grows *no faster* than  $g$  asymptotically.

#### Definition 2.1.2: Asymptotic Lower Bound ( $\Omega$ )

$f(n) = \Omega(g(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 1$  such that for all  $n \geq n_0$ :

$$f(n) \geq c \cdot g(n)$$

Informally:  $f$  grows *at least as fast* as  $g$ .

#### Definition 2.1.3: Tight Bound ( $\Theta$ )

$f(n) = \Theta(g(n))$  if  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$ . Informally:  $f$  and  $g$  have the *same* asymptotic growth rate.

### Complexity Hierarchy

| Class         | Name         | Example algorithm                           |
|---------------|--------------|---------------------------------------------|
| $O(1)$        | Constant     | Array access, hash table lookup             |
| $O(\log n)$   | Logarithmic  | Binary search                               |
| $O(n)$        | Linear       | Linear scan, linear search                  |
| $O(n \log n)$ | Linearithmic | Merge sort, Heap sort                       |
| $O(n^2)$      | Quadratic    | Insertion sort (worst), Bubble sort         |
| $O(n^3)$      | Cubic        | Naive matrix multiplication, Floyd-Warshall |
| $O(2^n)$      | Exponential  | Brute-force subset enumeration              |
| $O(n!)$       | Factorial    | Brute-force TSP                             |

**Remark.**

Growth order:  $1 \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n!$

**Rules for Computing Complexity**

- **Sum rule:**  $O(f) + O(g) = O(\max(f, g))$  — keep the dominant term.
- **Product rule:**  $O(f) \cdot O(g) = O(f \cdot g)$  — nested loops multiply.
- **Drop constants:**  $O(5n) = O(n)$ .
- **Drop lower-order terms:**  $O(n^2 + n) = O(n^2)$ .
- **Logarithm base:**  $O(\log_a n) = O(\log_b n)$  — base doesn't matter.

**2.2 Recursion****Definition 2.2.1: Recurrence Relation**

An equation defining  $T(n)$  in terms of  $T$  on smaller inputs. Typically  $T(n) = aT(n/b) + f(n)$  for divide-and-conquer algorithms, where  $a$  = number of subproblems,  $b$  = factor by which input shrinks,  $f(n)$  = cost outside recursive calls.

**Theorem 2.2.2: Master Theorem**

Let  $T(n) = aT(n/b) + f(n)$  with  $a \geq 1$ ,  $b > 1$ . Let  $d = \log_b a$ .

1. If  $f(n) = O(n^{d-\epsilon})$  for some  $\epsilon > 0$ :  $T(n) = \Theta(n^d)$
2. If  $f(n) = \Theta(n^d)$ :  $T(n) = \Theta(n^d \log n)$
3. If  $f(n) = \Omega(n^{d+\epsilon})$  and regularity holds:  $T(n) = \Theta(f(n))$

**Example 2.2.1**

**Merge Sort:**  $T(n) = 2T(n/2) + O(n)$ . Here  $a = 2$ ,  $b = 2$ ,  $d = \log_2 2 = 1$ . Since  $f(n) = O(n) = O(n^1) = \Theta(n^d)$ , Case 2 applies:  $T(n) = \Theta(n \log n)$ .

**Binary Search:**  $T(n) = T(n/2) + O(1)$ . Here  $a = 1$ ,  $b = 2$ ,  $d = \log_2 1 = 0$ . Since  $f(n) = O(1) = \Theta(n^0) = \Theta(n^d)$ , Case 2 applies:  $T(n) = \Theta(\log n)$ .

## Chapter 3

# Core Data Structures

### 3.1 Arrays

#### Definition 3.1.1: Array

A contiguous block of memory storing elements of the same type. Supports  $O(1)$  random access via index arithmetic. Python's `list` is a *dynamic array*.

| Operation                | Description             | Static | Dynamic  |
|--------------------------|-------------------------|--------|----------|
| Access <code>a[i]</code> | Read/write by index     | $O(1)$ | $O(1)$   |
| Search                   | Find element (unsorted) | $O(n)$ | $O(n)$   |
| Insert at end            | Append                  | N/A    | $O(1)^*$ |
| Insert at index $i$      | Shift elements right    | $O(n)$ | $O(n)$   |
| Delete at end            | Pop                     | N/A    | $O(1)^*$ |
| Delete at index $i$      | Shift elements left     | $O(n)$ | $O(n)$   |

#### Remark.

\* Amortized. A dynamic array doubles its capacity when full, costing  $O(n)$  occasionally, but the amortized cost per append is  $O(1)$ .

### 3.2 Stacks

#### Definition 3.2.1: Stack

A LIFO (Last-In, First-Out) abstract data type supporting: **push** (add to top), **pop** (remove from top), **peek** (read top). All operations in  $O(1)$ . Implemented with a list in Python.

| Operation | Python                   | Time     |
|-----------|--------------------------|----------|
| Push      | <code>s.append(x)</code> | $O(1)^*$ |
| Pop       | <code>s.pop()</code>     | $O(1)^*$ |
| Peek      | <code>s[-1]</code>       | $O(1)$   |
| Is empty  | <code>len(s) == 0</code> | $O(1)$   |

**Remark.**

- Classic applications: function call stack, expression parsing (balanced parentheses), DFS, undo/redo, monotonic stack problems.

### 3.3 Queues

**Definition 3.3.1: Queue**

A FIFO (First-In, First-Out) ADT supporting: **enqueue** (add to back), **dequeue** (remove from front), **peek**. Use `collections.deque` in Python —  $O(1)$  both ends. Using a plain `list` gives  $O(n)$  dequeue.

| Operation  | Python (deque)               | Time   |
|------------|------------------------------|--------|
| Enqueue    | <code>q.append(x)</code>     | $O(1)$ |
| Dequeue    | <code>q.popleft()</code>     | $O(1)$ |
| Peek front | <code>q[0]</code>            | $O(1)$ |
| Peek back  | <code>q[-1]</code>           | $O(1)$ |
| Push left  | <code>q.appendleft(x)</code> | $O(1)$ |
| Pop right  | <code>q.pop()</code>         | $O(1)$ |

**Remark.**

Applications: BFS, task scheduling, sliding window problems (monotonic deque). `collections.deque` is a doubly-linked list under the hood.

### 3.4 Linked Lists

**Definition 3.4.1: Linked List**

A sequence of *nodes*, each storing a value and a pointer to the next node. No random access; traversal is  $O(n)$ .

| Operation          | Description                | Singly         | Doubly |
|--------------------|----------------------------|----------------|--------|
| Access by index    | Traverse from head         | $O(n)$         | $O(n)$ |
| Search             | Linear scan                | $O(n)$         | $O(n)$ |
| Insert at head     | Update head pointer        | $O(1)$         | $O(1)$ |
| Insert at tail     | Requires tail pointer      | $O(1)^*$       | $O(1)$ |
| Insert at position | Traverse then insert       | $O(n)$         | $O(n)$ |
| Delete at head     | Update head pointer        | $O(1)$         | $O(1)$ |
| Delete at tail     | Traverse to second-to-last | $O(n)$         | $O(1)$ |
| Delete given node  | Unlink from neighbours     | $O(n)^\dagger$ | $O(1)$ |

**Remark.**

\* Requires a maintained tail pointer.  $^\dagger$  Need to find predecessor; in a doubly linked list, given the node, deletion is  $O(1)$ .

| Feature              | Array      | Linked List                       |
|----------------------|------------|-----------------------------------|
| Access by index      | $O(1)$     | $O(n)$                            |
| Insert/delete head   | $O(n)$     | $O(1)$                            |
| Insert/delete middle | $O(n)$     | $O(n)$ search + $O(1)$            |
| Memory               | Contiguous | Non-contiguous (pointer overhead) |
| Cache friendly       | Yes        | No                                |

### 3.5 Trees

#### Definition 3.5.1: Binary Tree

A rooted tree where each node has at most two children (left and right). Height  $h$  of a tree is the length of the longest root-to-leaf path. A *complete* binary tree has all levels full except possibly the last. A *full* binary tree has every node with 0 or 2 children.

#### Definition 3.5.2: Binary Search Tree (BST)

A binary tree where for every node  $v$ :

- all keys in the *left* subtree are **less than**  $v.key$
- all keys in the *right* subtree are **greater than**  $v.key$

Average case:  $O(\log n)$  for search, insert, delete. Worst case (skewed):  $O(n)$ .

#### Tree Traversals

| Traversal   | Order                           | Visit pattern     | Common use                |
|-------------|---------------------------------|-------------------|---------------------------|
| Inorder     | $L \rightarrow N \rightarrow R$ | Left, root, right | BST sorted output         |
| Preorder    | $N \rightarrow L \rightarrow R$ | Root, left, right | Copy tree, prefix expr    |
| Postorder   | $L \rightarrow R \rightarrow N$ | Left, right, root | Delete tree, postfix expr |
| Level-order | by depth                        | BFS row by row    | Level-by-level processing |

#### BST Operations

| Operation | Average     | Worst (skewed) |
|-----------|-------------|----------------|
| Search    | $O(\log n)$ | $O(n)$         |
| Insert    | $O(\log n)$ | $O(n)$         |
| Delete    | $O(\log n)$ | $O(n)$         |
| Min / Max | $O(\log n)$ | $O(n)$         |

#### Remark.

Self-balancing BSTs (AVL, Red-Black) guarantee  $O(\log n)$  worst case. Python's `heapq` implements a min-heap (complete binary tree stored as array).

# Chapter 4

## Sorting

### 4.1 Comparison-Based Sorting Algorithms

#### Insertion Sort

**Definition 4.1.1: Insertion Sort**

Build a sorted prefix one element at a time. For each element, shift it leftward into its correct position in the sorted portion.

---

**Algorithm 1** Insertion Sort

---

```
1: for  $i = 1$  to  $n - 1$  do
2:    $key \leftarrow A[i]$ 
3:    $j \leftarrow i - 1$ 
4:   while  $j \geq 0$  and  $A[j] > key$  do
5:      $A[j + 1] \leftarrow A[j]$ 
6:      $j \leftarrow j - 1$ 
7:   end while
8:    $A[j + 1] \leftarrow key$ 
9: end for
```

---

#### Merge Sort

**Definition 4.1.2: Merge Sort**

Divide array in half, recursively sort each half, merge the two sorted halves. Recurrence:  $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$ .

**Algorithm 2** Merge Sort

---

```

1: procedure MERGESORT( $A, l, r$ )
2:   if  $l < r$  then
3:      $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
4:     MERGESORT( $A, l, m$ )
5:     MERGESORT( $A, m + 1, r$ )
6:     MERGE( $A, l, m, r$ )
7:   end if
8: end procedure

```

---

**Remark.**

**Merge** takes two sorted subarrays and combines them in  $O(n)$  by comparing elements front-to-back. Requires  $O(n)$  auxiliary space.

**Quick Sort****Definition 4.1.3: Quick Sort**

Choose a *pivot*, partition the array so all elements  $\leq$  pivot are to its left and all  $>$  pivot to its right, then recurse on each side.

**Algorithm 3** Quick Sort

---

```

1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l < r$  then
3:      $p \leftarrow \text{PARTITION}(A, l, r)$ 
4:     QUICKSORT( $A, l, p - 1$ )
5:     QUICKSORT( $A, p + 1, r$ )
6:   end if
7: end procedure

```

---

**Remark.**

Partition (Lomuto scheme): pick last element as pivot; maintain index of “smaller” region. Average  $O(n \log n)$ ; worst case (already sorted)  $O(n^2)$ . Randomised pivot makes worst case extremely unlikely.

**Heap Sort****Definition 4.1.4: Heap Sort**

Build a max-heap from the array ( $O(n)$ ), then repeatedly extract the maximum and place it at the end ( $O(n \log n)$ ). Total:  $O(n \log n)$ , in-place, *not* stable.

**Selection Sort**

Find the minimum of the unsorted portion and swap it to the front. Always  $O(n^2)$ , regardless of input. *Not* stable.

## 4.2 Lower Bound for Comparison Sorting

### Theorem 4.2.1: $\Omega(n \log n)$ Lower Bound

Any comparison-based sorting algorithm requires  $\Omega(n \log n)$  comparisons in the worst case.

#### Proof

Model any comparison sort as a *decision tree*: each internal node is a comparison  $A[i] \leq A[j]$  (yes/no), and each leaf is a permutation of the sorted output. A correct algorithm must be able to produce all  $n!$  orderings, so the decision tree has  $\geq n!$  leaves. A binary tree of height  $h$  has at most  $2^h$  leaves, so  $2^h \geq n!$ , giving  $h \geq \log_2(n!) = \Omega(n \log n)$  (by Stirling's approximation).  $\square$

## 4.3 Sorting Algorithms Summary

| Algorithm      | Best          | Average       | Worst         | Space       | Stable |
|----------------|---------------|---------------|---------------|-------------|--------|
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | No     |
| Counting Sort  | $O(n + k)$    | $O(n + k)$    | $O(n + k)$    | $O(k)$      | Yes    |
| Radix Sort     | $O(d(n + k))$ | $O(d(n + k))$ | $O(d(n + k))$ | $O(n + k)$  | Yes    |

#### Remark.

Counting and Radix Sort are *not* comparison-based; they beat the  $\Omega(n \log n)$  lower bound by exploiting key structure.  $k$  = key range,  $d$  = number of digits.



# Chapter 5

## Graph Algorithms

### 5.1 Graph Basics

#### Definition 5.1.1: Graph

$G = (V, E)$  where  $V$  is a set of vertices and  $E \subseteq V \times V$  is a set of edges. A graph is *directed* (digraph) if edges have orientation, *undirected* otherwise.  $n = |V|$ ,  $m = |E|$ .

| Representation   | Description                                                 | Space      | Edge check  |
|------------------|-------------------------------------------------------------|------------|-------------|
| Adjacency Matrix | $n \times n$ boolean matrix; $M[u][v]=1$ if edge exists     | $O(n^2)$   | $O(1)$      |
| Adjacency List   | Array of lists; $\text{adj}[u]$ = list of neighbours of $u$ | $O(n + m)$ | $O(\deg u)$ |

#### Remark.

Use adjacency lists for sparse graphs ( $m \ll n^2$ ), adjacency matrix for dense graphs or when  $O(1)$  edge queries are required.

### 5.2 Breadth-First Search (BFS)

#### Definition 5.2.1: BFS

Explore all vertices at distance  $d$  from source before distance  $d + 1$ . Uses a queue. Computes shortest-path distances (unweighted) from source.

**Algorithm 4** BFS

---

```

1: procedure BFS( $G, s$ )
2:   Mark all vertices unvisited;  $\text{dist}[s] \leftarrow 0$ 
3:    $Q \leftarrow$  empty queue; enqueue  $s$ ; mark  $s$  visited
4:   while  $Q$  not empty do
5:      $u \leftarrow Q.\text{DEQUEUE}()$ 
6:     for each neighbour  $v$  of  $u$  do
7:       if  $v$  not visited then
8:         mark  $v$  visited;  $\text{dist}[v] \leftarrow \text{dist}[u] + 1$ 
9:          $Q.\text{ENQUEUE}(v)$ 
10:      end if
11:    end for
12:  end while
13: end procedure

```

---

Time:  $O(n + m)$ . Space:  $O(n)$  for queue and visited array.

### 5.3 Depth-First Search (DFS) & Applications

#### Definition 5.3.1: DFS

Explore as deep as possible before backtracking. Uses a stack (or recursion). Produces DFS tree with *tree*, *back*, *forward*, and *cross* edges (directed graphs).

**Algorithm 5** DFS

---

```

1: procedure DFS-VISIT( $G, u$ )
2:   mark  $u$  visited; push  $u$  onto stack (for topological order)
3:   for each neighbour  $v$  of  $u$  do
4:     if  $v$  not visited then
5:       DFS-VISIT( $G, v$ )
6:     end if
7:   end for
8:   record finish time of  $u$ 
9: end procedure

```

---

Time:  $O(n + m)$ .

---

| Edge type    | Meaning                                                          |
|--------------|------------------------------------------------------------------|
| Tree edge    | $v$ first discovered via $u$                                     |
| Back edge    | $v$ is an ancestor of $u$ (indicates a cycle in directed graphs) |
| Forward edge | $v$ is a descendant of $u$ but not through tree edge             |
| Cross edge   | $v$ is in a different subtree                                    |

---

## 5.4 Topological Sort

### Definition 5.4.1: Topological Sort

A linear ordering of vertices of a *DAG* (directed acyclic graph) such that for every edge  $(u, v)$ ,  $u$  appears before  $v$ .

**DFS-based algorithm:** Run DFS; output vertices in decreasing finish time.  $O(n + m)$ .

**Kahn's algorithm (BFS-based):** Repeatedly remove vertices with in-degree 0.  $O(n + m)$ .

**Remark.**

A directed graph has a topological sort *if and only if* it is a DAG (no back edges in DFS).

## 5.5 Strongly Connected Components (SCC)

### Definition 5.5.1: SCC

A maximal set of vertices  $C \subseteq V$  such that every vertex in  $C$  is reachable from every other vertex in  $C$ .

**Kosaraju's Algorithm** ( $O(n + m)$ ):

1. Run DFS on  $G$ ; record finish times.
2. Compute  $G^T$  (transpose: reverse all edges).
3. Run DFS on  $G^T$  in decreasing finish-time order; each DFS tree is one SCC.

**Tarjan's Algorithm** ( $O(n + m)$ ): Single DFS with a stack; uses *discovery times* and *low-link values* to identify SCCs.

## 5.6 Minimum Spanning Trees (MST)

### Definition 5.6.1: Minimum Spanning Tree

Given a connected, undirected, weighted graph  $G = (V, E, w)$ , an MST is a spanning tree  $T \subseteq E$  that minimises  $\sum_{e \in T} w(e)$ . Any spanning tree has exactly  $n - 1$  edges.

### Kruskal's Algorithm

Sort all edges by weight; add each edge if it doesn't form a cycle (use Union-Find).

**Algorithm 6** Kruskal's MST

---

```

1: Sort edges:  $e_1 \leq e_2 \leq \dots \leq e_m$ 
2:  $T \leftarrow \emptyset$ ; initialise Union-Find on  $V$ 
3: for each edge  $(u, v)$  in sorted order do
4:   if  $\text{FIND}(u) \neq \text{FIND}(v)$  then
5:      $T \leftarrow T \cup \{(u, v)\}$ 
6:      $\text{UNION}(u, v)$ 
7:   end if
8: end for
9: return  $T$ 

```

---

Time:  $O(m \log m)$  dominated by sorting (or  $O(m\alpha(n))$  with path compression).

**Prim's Algorithm**

Grow the MST greedily from a starting vertex; always add the minimum-weight edge crossing the cut between tree and non-tree vertices.

Time:  $O(m \log n)$  with a binary heap;  $O(m + n \log n)$  with a Fibonacci heap.

**5.7 Shortest Paths****Single-Source Shortest Paths****Definition 5.7.1: Dijkstra's Algorithm**

Greedy SSSP for graphs with *non-negative* edge weights. Maintains a priority queue of (distance, vertex) pairs.  
Time:  $O((n + m) \log n)$  with a binary heap.

**Algorithm 7** Dijkstra's Algorithm

---

```

1:  $\text{dist}[s] \leftarrow 0$ ;  $\text{dist}[v] \leftarrow \infty$  for all  $v \neq s$ 
2: Priority queue  $PQ \leftarrow \{(0, s)\}$ 
3: while  $PQ$  not empty do
4:    $(d, u) \leftarrow PQ.\text{EXTRACTMIN}()$ 
5:   if  $d > \text{dist}[u]$  then continue
6:   end if
7:   for each edge  $(u, v, w)$  do
8:     if  $\text{dist}[u] + w < \text{dist}[v]$  then
9:        $\text{dist}[v] \leftarrow \text{dist}[u] + w$ 
10:       $PQ.\text{INSERT}((\text{dist}[v], v))$ 
11:     end if
12:   end for
13: end while

```

---

**Definition 5.7.2: Bellman-Ford**

SSSP for graphs with *negative* edge weights (no negative cycles). Relax all  $m$  edges  $n - 1$  times. Time:  $O(nm)$ .  
Detects negative cycles: if any edge can be relaxed after  $n - 1$  rounds, a negative cycle is reachable from  $s$ .

## All-Pairs Shortest Paths

**Definition 5.7.3: Floyd-Warshall**

Computes shortest paths between *all* pairs of vertices. DP approach:  $d^{(k)}[i][j]$  = shortest path from  $i$  to  $j$  using only vertices  $\{1, \dots, k\}$  as intermediates.

$$d^{(k)}[i][j] = \min\left(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\right)$$

Time:  $O(n^3)$ . Space:  $O(n^2)$ .

| Algorithm      | Scope | Weights            | Time                | Space    |
|----------------|-------|--------------------|---------------------|----------|
| BFS            | SSSP  | Unweighted         | $O(n + m)$          | $O(n)$   |
| Dijkstra       | SSSP  | $\geq 0$           | $O((n + m) \log n)$ | $O(n)$   |
| Bellman-Ford   | SSSP  | Any (no neg cycle) | $O(nm)$             | $O(n)$   |
| Floyd-Warshall | APSP  | Any (no neg cycle) | $O(n^3)$            | $O(n^2)$ |

## Chapter 6

# Algorithm Design Paradigms

### 6.1 Divide & Conquer

#### Definition 6.1.1: Divide & Conquer

Recursively break a problem into  $a$  subproblems of size  $n/b$ , solve each, then combine. Cost:  $T(n) = aT(n/b) + f(n)$ .

#### Closest Pair of Points

**Problem:** Given  $n$  points in the plane, find the pair with minimum Euclidean distance. Naive brute-force:  $O(n^2)$ .

**Algorithm** ( $O(n \log n)$ ):

1. Sort points by  $x$ -coordinate.
2. Divide at median  $x$ ; recursively find  $\delta_L$  and  $\delta_R$ .
3. Let  $\delta = \min(\delta_L, \delta_R)$ .
4. **Combine:** Consider the vertical strip of width  $2\delta$  around the dividing line. For each point in the strip (sorted by  $y$ ), check at most **7** subsequent points — the geometry bounds the density.
5. Return  $\min(\delta, \delta_{\text{strip}})$ .

#### Theorem 6.1.2: Closest Pair Complexity

$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$  by Master Theorem Case 2.

### 6.2 Greedy Algorithms

#### Definition 6.2.1: Greedy Algorithm

Make the locally optimal choice at each step, hoping to reach a global optimum. Correctness requires an *exchange argument* or *greedy stays ahead* proof.

## Interval Scheduling

**Problem:** Given  $n$  intervals  $[s_i, f_i]$ , select the maximum number of non-overlapping intervals.

**Algorithm:** Sort by finish time  $f_i$ ; greedily add intervals that start after the last selected one finishes.

### Theorem 6.2.2: Interval Scheduling Greedy is Optimal

**Proof (Exchange argument):** Let  $G = \{g_1, g_2, \dots, g_k\}$  be greedy solution,  $O = \{o_1, o_2, \dots, o_m\}$  an optimal solution, both sorted by finish time. Show by induction:  $f(g_i) \leq f(o_i)$  for all  $i$ . Since greedy finishes no later than optimal at each step, it can always fit at least as many intervals. Hence  $k \geq m$ , so  $k = m$ .

## Huffman Coding

**Problem:** Build an optimal prefix-free binary code minimising total encoded length, given symbol frequencies.

**Algorithm:** Use a min-heap; repeatedly merge the two nodes with lowest frequency into a new node with their combined frequency.

Time:  $O(n \log n)$ .

**Remark.**

■ Huffman coding is provably optimal for prefix-free codes.

## 6.3 Dynamic Programming

### Definition 6.3.1: Dynamic Programming

Solve a problem by breaking it into overlapping subproblems, storing results (*memoisation* or bottom-up table), and combining. Applicable when the problem has: (1) **Optimal substructure:** optimal solution contains optimal sub-solutions. (2) **Overlapping subproblems:** subproblems recur.

### Longest Common Subsequence (LCS)

**Problem:** Given strings  $X[1..m]$  and  $Y[1..n]$ , find the longest subsequence common to both.

**Recurrence:**

$$\text{dp}[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ \text{dp}[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(\text{dp}[i-1][j], \text{dp}[i][j-1]) & \text{otherwise} \end{cases}$$

Time:  $O(mn)$ . Space:  $O(mn)$  (reducible to  $O(\min(m, n))$ ).

### 0/1 Knapsack

**Problem:**  $n$  items each with weight  $w_i$  and value  $v_i$ ; knapsack capacity  $W$ . Maximise total value with total weight  $\leq W$ .

**Recurrence:**

$$\text{dp}[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \text{dp}[i-1][w] & \text{if } w_i > w \\ \max(\text{dp}[i-1][w], v_i + \text{dp}[i-1][w - w_i]) & \text{otherwise} \end{cases}$$

Time:  $O(nW)$ . Space:  $O(nW)$  (reducible to  $O(W)$ ).

**Remark.**

- This is *pseudo-polynomial*: exponential in the input size (bits of  $W$ ) but polynomial in the *value*  $W$ .

**Weighted Interval Scheduling (Interval DP)**

**Problem:** Select non-overlapping intervals to maximise total weight.

Sort intervals by finish time. Let  $p(i) = \text{largest } j < i \text{ with } f_j \leq s_i$ .

$$\text{dp}[i] = \max(\text{dp}[i-1], w_i + \text{dp}[p(i)])$$

Time:  $O(n \log n)$  (binary search for  $p(i)$ ).

**6.4 Matching****Bipartite Matching****Definition 6.4.1: Bipartite Graph**

$G = (U \cup V, E)$  where all edges go between  $U$  and  $V$ . A *matching*  $M \subseteq E$  is a set of edges with no shared endpoints. A *maximum matching* maximises  $|M|$ .

**Algorithm** (Augmenting Paths, Hopcroft-Karp): Find shortest augmenting paths (paths from unmatched  $U$  to unmatched  $V$  alternating matched/unmatched edges). Time:  $O(\sqrt{n} \cdot m)$ .

**Stable Matching (Gale-Shapley)**

**Problem:** Given  $n$  men and  $n$  women each with preference rankings, find a stable matching (no blocking pair: a man-woman pair who both prefer each other over current partners).

**Algorithm 8 Gale-Shapley (Propose-and-Reject)**


---

```

1: while any man is unmatched do
2:    $m$  proposes to his highest-ranked woman  $w$  not yet proposed to
3:   if  $w$  is unmatched then
4:     tentatively match  $(m, w)$ 
5:   else if  $w$  prefers  $m$  over current partner  $m'$  then
6:     unmatched  $m'$ ; tentatively match  $(m, w)$ 
7:   end if
8: end while

```

---

**Theorem 6.4.2: Gale-Shapley Correctness**

The algorithm terminates in  $O(n^2)$  and produces a *stable* matching. The result is **man-optimal** (each man gets his best stable partner) and **woman-pessimal** (each woman gets her worst stable partner).



## Chapter 7

# Computability & Complexity

### 7.1 Computability

#### Definition 7.1.1: Turing Machine

A mathematical model of computation consisting of an infinite tape, a read/write head, a finite set of states, and a transition function. Defines what is *computable* in principle.

#### Definition 7.1.2: Decidable Language

A language  $L$  is *decidable* (recursive) if there exists a Turing machine that *always halts* and correctly accepts  $x \in L$  and rejects  $x \notin L$ .

#### Definition 7.1.3: Recognisable Language

A language  $L$  is *recognisable* (recursively enumerable) if there exists a Turing machine that accepts  $x \in L$  (but may loop on  $x \notin L$ ).

#### Theorem 7.1.4: Halting Problem is Undecidable

Let  $\text{HALT} = \{\langle M, x \rangle \mid M \text{ halts on input } x\}$ .  $\text{HALT}$  is not decidable.

#### Proof sketch

Assume for contradiction a decider  $H$  for  $\text{HALT}$  exists. Construct  $D$ : on input  $\langle M \rangle$ , run  $H(\langle M, \langle M \rangle \rangle)$ ; if  $H$  accepts (i.e.  $M$  halts on  $\langle M \rangle$ ), then  $D$  loops; if  $H$  rejects,  $D$  halts. Evaluate  $D$  on  $\langle D \rangle$ : contradiction in both cases. Hence  $H$  cannot exist.  $\square$

## 7.2 NP-Completeness

### Definition 7.2.1: Complexity Classes

- **P**: Decision problems solvable in polynomial time.
- **NP**: Decision problems with solutions *verifiable* in polynomial time. Equivalently, solvable by a nondeterministic TM in polynomial time.
- **NP-hard**: At least as hard as every problem in NP (via polynomial-time many-one reductions). Need not be in NP.
- **NP-complete**: In NP *and* NP-hard.

#### Remark.

$\mathbf{P} \subseteq \mathbf{NP}$ . Whether  $\mathbf{P} = \mathbf{NP}$  is the central open problem of computer science.

### Definition 7.2.2: Polynomial-Time Reduction

$A \leq_p B$  (“ $A$  reduces to  $B$ ”) if there exists a polynomial-time computable function  $f$  such that  $x \in A \iff f(x) \in B$ . If  $B \in P$  then  $A \in P$ ; contrapositively, if  $A \notin P$  then  $B \notin P$ .

### Theorem 7.2.3: Cook-Levin Theorem

3-SAT is NP-complete. (Every NP problem reduces to 3-SAT.)

## Classic NP-Complete Problems

| Problem          | Description                                                     |
|------------------|-----------------------------------------------------------------|
| 3-SAT            | Boolean formula in 3-CNF; satisfying assignment?                |
| INDEPENDENT SET  | Graph $G$ , integer $k$ ; independent set of size $\geq k$ ?    |
| CLIQUE           | Graph $G$ , integer $k$ ; clique of size $\geq k$ ?             |
| VERTEX COVER     | Graph $G$ , integer $k$ ; vertex cover of size $\leq k$ ?       |
| 3-COLORING       | Can vertices of $G$ be colored with 3 colors, no adjacent same? |
| HAMILTONIAN PATH | Does $G$ have a path visiting every vertex exactly once?        |
| TSP (DECISION)   | Tour visiting all cities with total cost $\leq k$ ?             |
| SUBSET SUM       | Integers $S$ , target $t$ ; subset summing to exactly $t$ ?     |

## Key Reductions

### 3-SAT $\leq_p$ Independent Set

**Construction:** Given a 3-CNF formula with  $k$  clauses, build graph  $G$ :

- For each clause, create a *triangle* (3 nodes, one per literal).
- Add an edge between any two nodes in different triangles representing *complementary* literals ( $x$  and  $\bar{x}$ ).

### Theorem 7.2.4: Reduction correctness

$\phi$  is satisfiable  $\iff G$  has an independent set of size  $k$ .

- $(\Rightarrow)$  A satisfying assignment selects one true literal per clause; these  $k$  nodes form an independent set (no edges within a triangle since we pick one; no inter-triangle edges since complementary literals can't both be true).
- $(\Leftarrow)$  An independent set of size  $k$  picks exactly one node per triangle; assign those literals true (no contradiction since complement pairs are excluded).

### Independent Set $\leq_p$ Vertex Cover

### Theorem 7.2.5: Complement duality

$S$  is an independent set of size  $k$  in  $G \iff V \setminus S$  is a vertex cover of size  $n - k$ .

#### Proof

$(\Rightarrow)$  For any edge  $(u, v)$ : since  $S$  is independent, not both  $u, v \in S$ , so at least one of  $u, v \in V \setminus S$ . Hence  $V \setminus S$  covers every edge.

$(\Leftarrow)$  Suppose  $u, v \in S$ . Then neither is in  $V \setminus S$ , but edge  $(u, v)$  is uncovered — contradiction. So  $S$  is independent.  $\square$

### 3-SAT $\leq_p$ 3-Coloring

**Construction:** Build a graph with a *palette* gadget (3 nodes: T, F, B pairwise connected) and a *clause gadget* for each clause that forces at least one literal to match T. The reduction preserves satisfiability.

### Coping with NP-Completeness

| Strategy                  | Description                                                                              |
|---------------------------|------------------------------------------------------------------------------------------|
| Special cases             | Some NP-complete problems are poly-time on restricted inputs (e.g. 2-SAT, planar graphs) |
| Approximation             | Guarantee solution within factor $\alpha$ of optimal                                     |
| FPT algorithms            | Exponential only in a small parameter $k$ (e.g. $O(2^k \cdot n)$ )                       |
| Heuristics / Local search | No guarantee, but fast in practice                                                       |